

Containerized Notes Application

Deployed with Docker, Kubernetes, and AWS Cloud

Submitted by:

Name: Himanshu Fulera

Registration No.: 12304291

**In partial fulfilment for the requirements of the award of the degree
of**

Bachelor of Technology(Computer Science)



L OVELY
P ROFESSIONAL
U NIVERSITY

**Lovely Professional University
Phagwara, Punjab**

1. Project Overview

The "Containerized Notes Application" is a robust, cloud-hosted web application engineered to demonstrate modern DevOps practices, container orchestration, and cloud infrastructure management. Developed using the Python Flask framework, the application provides a seamless interface for users to create, view, and manage personal notes.

Live Application Demo: <http://16.170.204.49:30007/>
(Note: Availability is contingent on the active state of the underlying AWS EC2 instance.)

1.1 Core Features

- **CRUD Operations:** Complete capability to Add, View, and Delete notes.
- **Responsive UI:** Frontend designed with HTML, CSS, and JS to adapt to various screen sizes.
- **Immutable Infrastructure:** Containerized environment ensuring code runs consistently.
- **Self-Healing Orchestration:** Kubernetes deployment ensuring pod restarts in the event of failure.

2. System Architecture & Workflow

The system is designed in a decoupled, layered architecture to isolate dependencies and improve scalability. The workflow begins at the developer level and extends out to the global cloud infrastructure.

2.1 Request Flow

1. **Client Request:** A user accesses the application via their web browser using the AWS EC2 Public IP.
2. **AWS Security Group:** The EC2 firewall evaluates the inbound request and permits traffic on port 30007.
3. **Kubernetes Service (NodePort):** The K8s service intercepts the request and routes it to an available Pod running within the Minikube cluster.
4. **Docker Container:** Inside the Pod, the Docker container executes the Flask WSGI server to process the HTTP request.
5. **Application Logic:** Flask interacts with the data state and renders the HTML templates.

3. Technologies and Tools Utilized

Technology	Role in Project	Technical Justification
Python / Flask	Backend Framework	Lightweight framework ideal for rapidly prototyping RESTful services.
Docker	Containerization Engine	Packages application & environment into a single executable artifact.
Docker Hub	Image Registry	Centralized repository for cloud deployment.
Kubernetes (Minikube)	Container Orchestration	Manages deployment, scaling, and networking of Docker containers.
AWS EC2	Cloud Infrastructure	Delivers scalable IaaS using an Ubuntu-based instance.

4. Implementation Phase & Execution

Step 1: Containerizing the Application (Docker)

```
docker build -t notes-app .  
docker run -d -p 5000:5000 notes-app
```

Step 2: Pushing to Registry

```
docker tag notes-app hkhimanshu3214/notes-app:latest  
docker push hkhimanshu3214/notes-app:latest
```

Step 3: AWS EC2 Provisioning

An m7i-flex.large instance was provisioned in eu-north-1. Security groups were updated to allow SSH and TCP (Port 30007).

Step 4: Kubernetes Orchestration

```
minikube start --driver=docker  
kubectl apply -f deployment.yaml  
kubectl apply -f service.yaml
```

5. System Screenshots & Analysis

5.1 Application User Interface (Homepage)

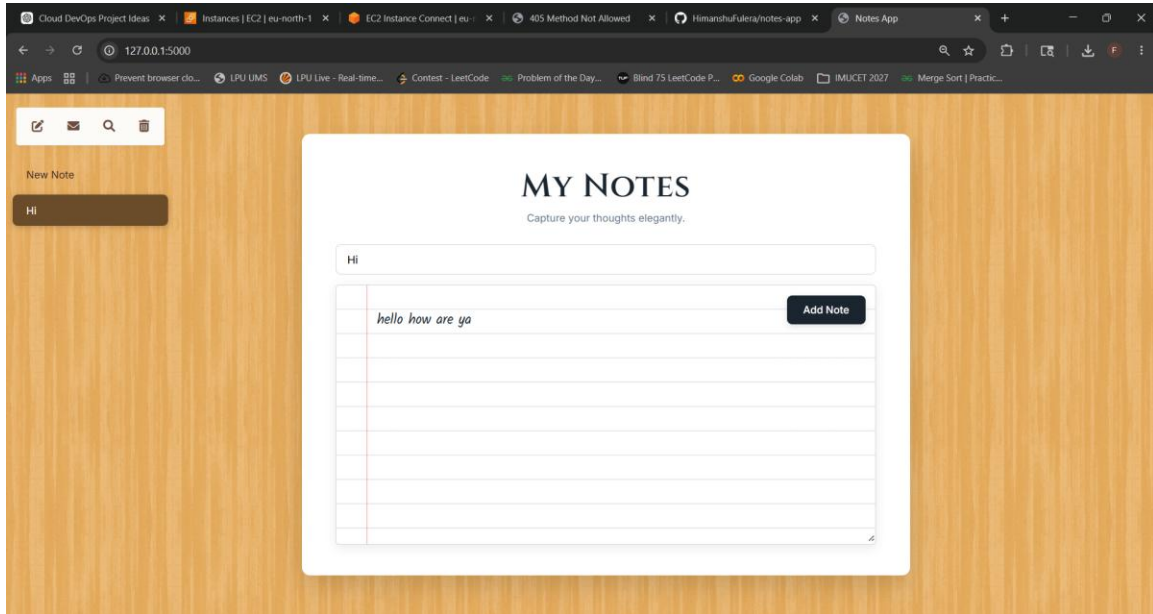


Figure: 5.1 Application User Interface (Homepage)

Hosted at 127.0.0.1:5000 during local testing, this showcases a clean UI with a wood-texture background. The input fields and rendered notes confirm the Flask backend is correctly processing form data.

5.2 Container Registry (Docker Hub)

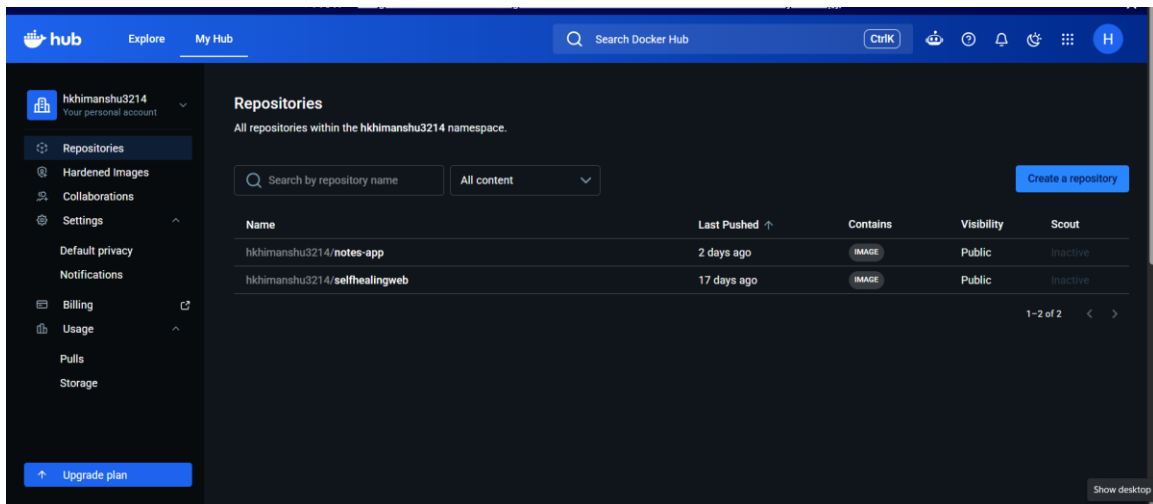


Figure: 5.2 Container Registry (Docker Hub)

The Docker Hub dashboard validates the successful push of the application image under the namespace hkhimanshu3214/notes-app.

5.3 Container Runtime Environment (Docker PS)

```
ubuntu@ip-172-31-38-248:~$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
af8ac704dd5a   hkhimanshu3214/notes-app           "python app.py"         2 days ago    Up 2 days    0.0.0.0:30007->5000/tcp, [::]:30007->5000/tcp
b168353503c6   gcr.io/k8s-minikube/kicbase:v0.0.50 "/usr/local/bin/entr..." 2 days ago    Up 2 days    127.0.0.1:32770->5000/tcp, 127.0.0.1:32771->8443/tcp, 127.0.0.1:32772->32443/tcp
minikube
```

Figure: 5.3 Container Runtime Environment (Docker PS)

Analysis: The execution of docker ps reveals the active container hkhimanshu3214/notes-app. The port mapping 0.0.0.0:30007->5000/tcp securely routes traffic to the Flask server.

5.4 Orchestration Layer (Kubernetes Pods & Services)

```
ubuntu@ip-172-31-38-248:~$ kubectl get nodes
NAME          STATUS    ROLES          AGE      VERSION
minikube      Ready     control-plane   2d5h     v1.35.1
ubuntu@ip-172-31-38-248:~$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
notes-app-deployment-5f7484b9b8-m92dz 1/1     Running   1 (98s ago) 2d5h
ubuntu@ip-172-31-38-248:~$ kubectl get svc
NAME            TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
kubernetes      ClusterIP     10.96.0.1      <none>          443/TCP           2d5h
notes-app-service NodePort      10.104.175.215 <none>          5000:30007/TCP   2d5h
ubuntu@ip-172-31-38-248:~$ kubectl get deployments
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
notes-app-deployment 1/1     1             1           2d5h
ubuntu@ip-172-31-38-248:~$
```

Figure: 5.4 Orchestration Layer (Kubernetes Pods & Services)

Analysis: The terminal output confirms Minikube control plane health. The notes-app-service is bridged via NodePort to external traffic on 30007.

5.5 Cloud Infrastructure (AWS EC2 Dashboard)

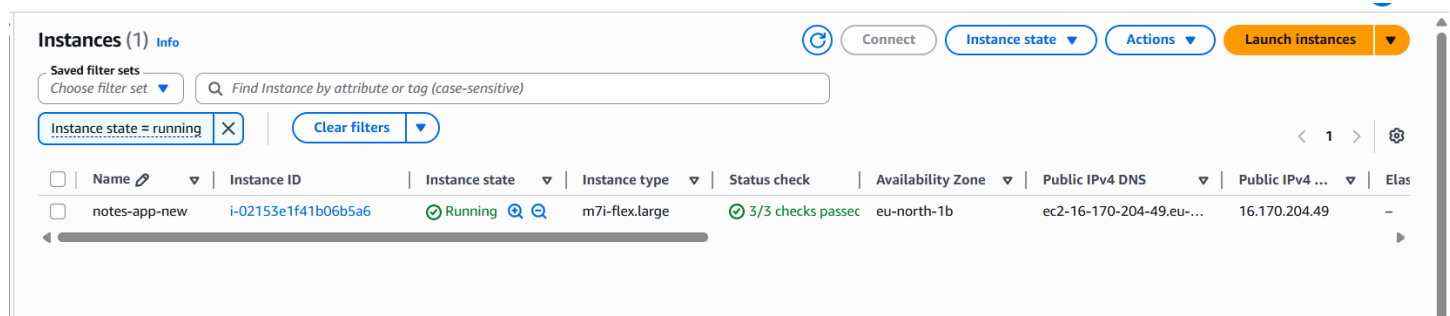


Figure: 5.5 Cloud Infrastructure (AWS EC2 Dashboard)

Analysis: The AWS console shows the notes-app-new instance utilizing m7i-flex.large compute with a Public IPv4 address 16.170.204.49.

6. Future Enhancements

- CI/CD Integration: Implement a Jenkins pipeline for automated deployment.
- Persistent Storage: Integrate AWS RDS or Kubernetes Persistent Volumes to prevent data loss.
- Domain Mapping & SSL: Map EC2 IPv4 to a custom domain using AWS Route53 and secure endpoints with HTTPS.

7. Conclusion

The Containerized Notes Application successfully bridges the gap between software development and IT operations. By integrating Docker for consistent artifact generation and Kubernetes for resilient deployment, the project guarantees zero-downtime potential and high scalability. Deploying the cluster on AWS EC2 provided hands-on exposure to cloud provisioning, network security groups, and public internet routing.